Chapter 3

Algorithm

- Algorithm
  - Series of actions in specific order
  - It is a procedure for solving a problem in terms of:
    - The actions executed
    - The order in which actions are to be executed
- Program control
  - Specifying the order in which actions execute
    - **Control structures** help specify this order

Pseudocode

- **Pseudocode**
  - Informal language for developing algorithms
  - Not executed on computers
  - Helps developers “think out” algorithms
  - A carefully prepared pseudocode can be easily converted to a Java program
  - Pseudocode normally describes only statements describing the actions:
    - Actions like input, output, calculations

Control Structures

- Sequential execution
  - Program statements execute one after the other
- Transfer of control
  - Various Java statements enable us to specify the next statement to execute which may not be the next one in sequence
  - Three control statements can specify order of statements
    - Sequence structure
    - Selection structure
    - Repetition structure
- Flowchart
  - Graphical representation of algorithm
    - Flowlines indicate order in which actions execute

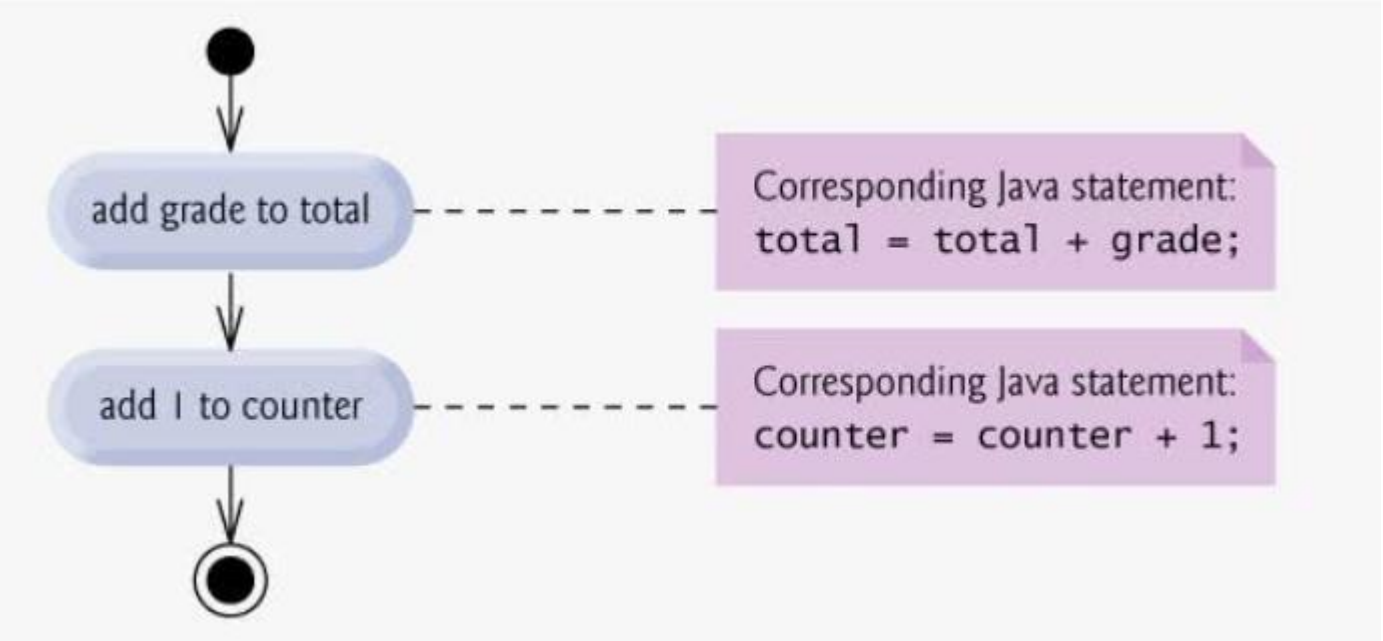
UML Activity Diagram

- UML Activity Diagram models the workflow (activity) of a portion of a software system
- Activity Diagrams are composed of symbols:
  - Action state symbol (rectangle)
  - Diamonds (decision)
  - Small circles
    - Solid circle – at the top represents beginning of the activity diagram
    - Solid circle inside a hollow circle at the end marks the end – **final state**
  - Rectangle with right upper corner folded – UML notes (like comments): are connected with action states with dotted lines
- These symbols are connected with transmission arrows, which represent flow of the activity
- Activity diagram help in developing and representing algorithms

Sequence Structure in Java

- Sequence structure is built into Java
- Unless directed, the computer executes Java statements one after the other in the order in which they are written
- This is called sequential structure
- Next slide shows UML diagram of sequence structure in which two simple calculations are done in order:
  - total = total + grade
  - counter = counter + 1

Activity Diagram – Sequence Structure



Selection Statements in Java

- Java provides three selection structures
  - **if** – performs an action if a condition is true, or skips it, if condition is false
    - **Called as: Single Selection statement**
  - **if/else :** performs an action if condition is true and performs a different action if the condition is false
    - **Called as : Double selection statement**
  - **switch:** performs one of many different actions, depending on value of an expression
    - **Called as: multiple selection statement**

Repetition Statements in Java

- Java provides three repetition structures
  - `while`
  - `do/while`
  - `for`
- Enables programs to perform statements repeatedly as long as a condition remains true.
- They are also called as repetition statements
- The while and for statements perform action in their bodies zero or more times (if the condition is initially false…the action will not perform
- The do..while statement performs the action in its body one or more times

The if Single Selection Structure

- Single-entry/single-exit structure
- Perform action only when condition is **`true`**
- Action/decision programming model
- Example : passing grade on an exam is 60
- Pseudocode:

  If student's grade is greater than or equal to 60

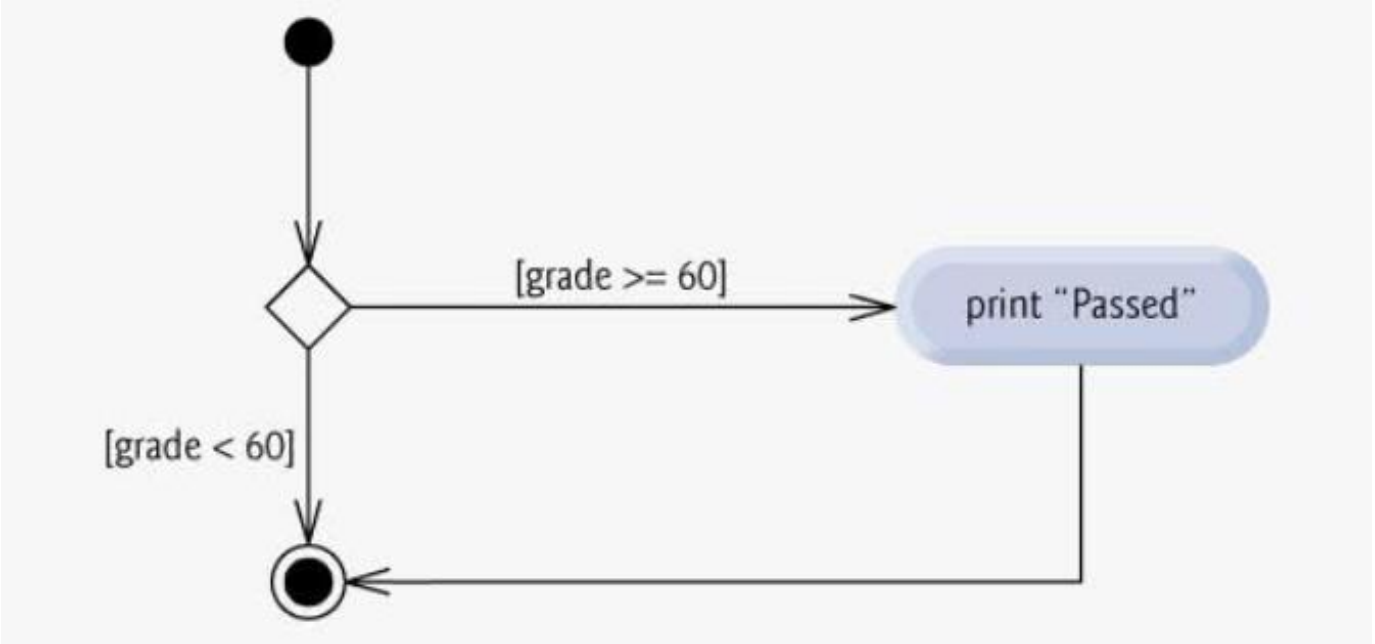
  print "Student Passed"

Program statement:

```
if ( studentGrade >= 60 )
        System.out.println( "Student Passed" );
```

The if Single Selection Structure

- This example determines if condition is true (student's grade is greater than or equal to 60),
- if so "student passed" is printed and next statement in the order is performed.
- If condition is false, print statement is ignored, and next statement in the order is performed.
- UML Activity Diagram :
- Condition – **Guard Condition**



if…else Double Selection Statement

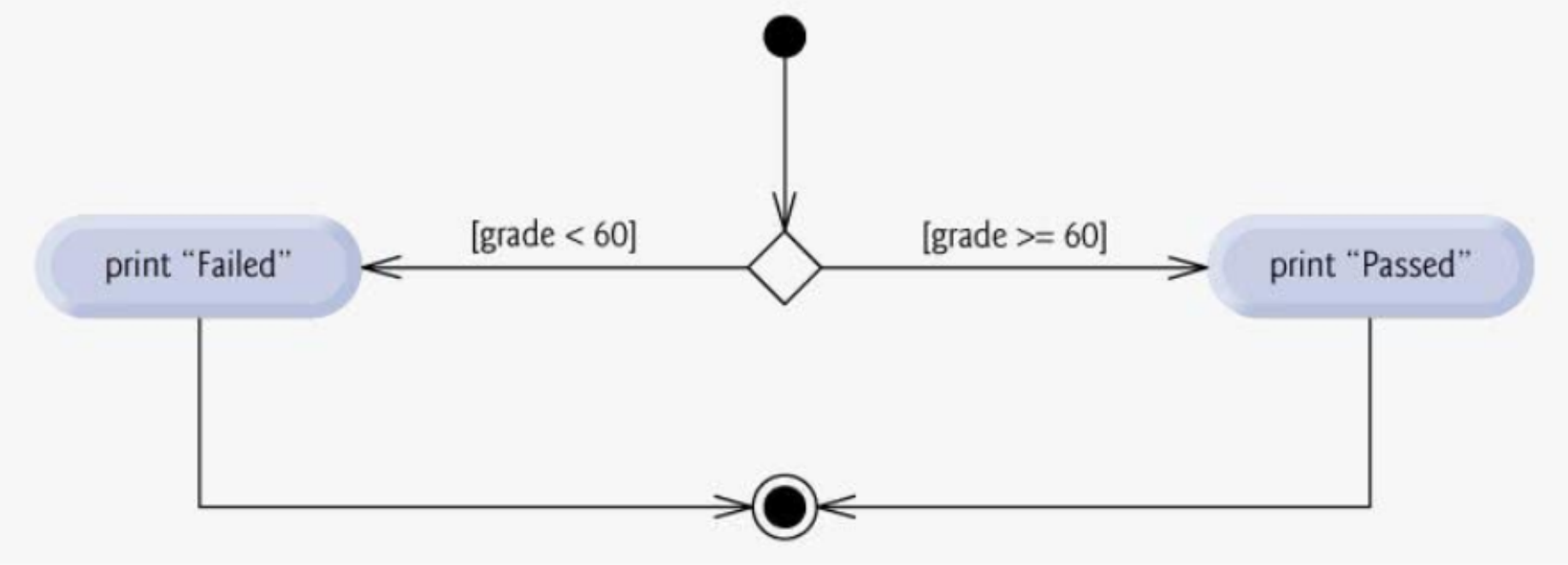
- If.. Else allows to specify an action to perform when the condition is true and different action when the condition is false.
- Example Pseudocode:

```
If student's grade is greater than or equal to 60
    Print "Passed"
Else
    Print "Failed"
```

- Corresponding Code:

```
if ( grade >= 60 )
        System.out.println( "Passed" );
else
        System.out.println( "Failed" );
```

if…else Double Selection Statement



Conditional Operator (?:)

- Can be used in place of if—else
- Java's only ternary operator
- Together, the operands and the ?: symbol form conditional expression
- First Operand : Boolean expression (evaluates to true or false)
- Second Operand: Value of expression if the condition is true
- Third Operand: Value of expression if the condition is false
- Example:

```
String result = studentGrade >= 60 ? "Passed" : "Failed";
System.out.println(result);
```

Nested if…else

- A program can test multiple cases by placing if...else statements inside other if...else statements to create **nested if...else statements**.
- For example, the following pseudocode represents a nested if...else that prints A for exam grades greater than or equal to 90, B for grades in the range 80 to 89, C for grades in the range 70 to 79, D for grades in the range 60 to 69 and F for all other grades:

```
If student's grade is greater than or equal to 90
        Print "A"
else
        If student's grade is greater than or equal to 80
                Print "B"
        else
                If student's grade is greater than or equal to 70
                        Print "C"
                else
                        If student's grade is greater than or equal to 60
                                Print "D"
                        else
                                Print "F"
```

Nested if…else

```
if ( studentGrade >= 90 )
        System.out.println( "A" );
else
        if ( studentGrade >= 80 )
                System.out.println( "B" );
        else
                if ( studentGrade >= 70 )
                        System.out.println( "C" );
                else
                        if ( studentGrade >= 60 )
                                System.out.println( "D" );
                        else
                                System.out.println( "F" );
```

Nested if…else

- Can be written as:

```
if ( studentGrade >= 90 )
        System.out.println( "A" );
else if ( studentGrade >= 80 )
        System.out.println( "B" );
else if ( studentGrade >= 70 )
        System.out.println( "C" );
else if ( studentGrade >= 60 )
        System.out.println( "D" );
else
        System.out.println( "F" );
```

Blocks

- The if statement normally expects only one statement in its body.
- To include several statements in the body of an if (or the body of an else for an if...else statement), enclose the statements in braces ({ and }).
- A set of statements contained within a pair of braces is called a **block**.

```
if ( grade >= 60 )
    {
        System.out.println( "Passed" );
        System.out.println(“Congratulation !!);
    }
else
    {
        System.out.println( "Failed" );
        System.out.println( "You must take this course again." );
    }
```

Dangling else Problem

- The Java compiler always associates an else with the immediately preceding if unless told to do otherwise by the placement of braces ({ and }).
- This behavior can lead to what is referred to as the **dangling-else problem**. For example,

```
if ( x > 5 )
    if ( y > 5 )
        System.out.println( "x and y are > 5" );
 else
       System.out.println( "x is <= 5" );
```

- appears to indicate that if x is greater than 5, the nested if statement determines whether y is also greater than 5.
- If so, the string "x and y are > 5" is output.
- Otherwise, it appears that if x is not greater than 5, the else part of the if...else outputs the string "x is <= 5".
- Beware!
- This nested if...else statement does not execute as it appears.

Dangling else Problem

- The compiler interprets as:

- The outer if statement tests whether x is greater than 5.
- If so, execution continues by testing whether y is also greater than 5.
- If the second condition is true, the proper string"x and y are > 5"is displayed.
- However, if the second condition is false, the string "x is <= 5" is displayed, even though we know that x is greater than 5.
- To force the nested if...else statement to execute as it was originally intended, we must write it as follows:

```
if ( x > 5 )
{
        if ( y > 5 )
                System.out.println( "x and y are > 5" );
}
else
                System.out.println( "x is <= 5" );
```

while Repetition Statement

- The repetition (or looping) statement allows to specify that a program should repeat an action while some condition remains **true.**

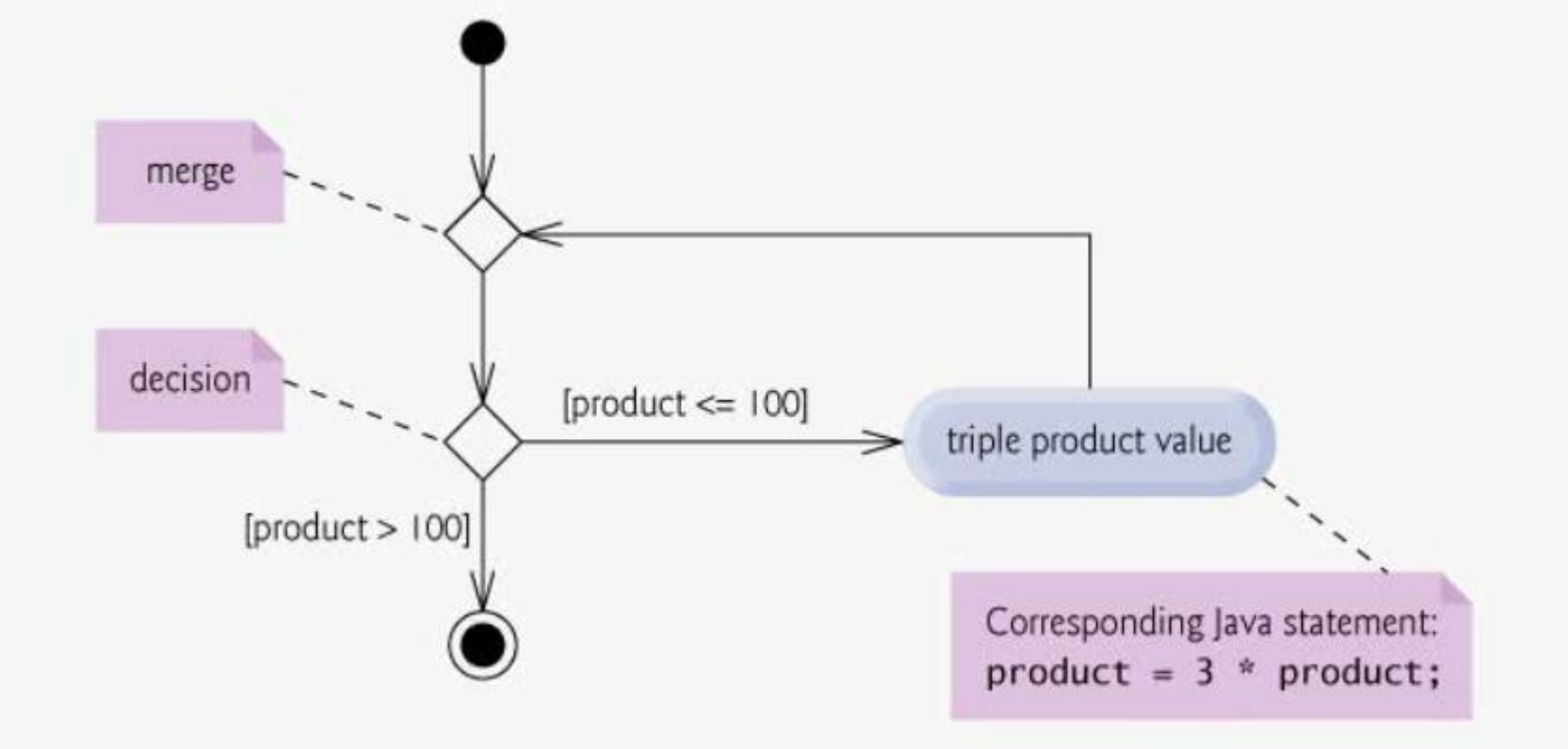
- As an example of Java's while repetition statement, consider a program segment designed to find the first power of 3 larger than 100.
- Suppose that the int variable product is initialized to 3.
- When the following while statement finishes executing, product contains the result:

```
int product = 3;
while ( product <= 100 )
        product = 3 * product;
```

- When this while statement begins execution, the value of variable product is 3.
- Each iteration of the while statement multiplies product by 3, so product takes on the values 9, 27, 81 and 243 successively.
- When variable product becomes 243, the while statement condition product <= 100 becomes false.
- This terminates the repetition, so the final value of product is 243.

while Repetition Statement

- UML Activity Diagram of Example



Example while loop Program

Sum of first 10 numbers

```
public class SumDigit
{
        public static void main (String args[])
        {
                int sum=0, count =1;
                while (count <=10)
                {
                        sum = sum + count;
                        count = count + 1;
                }
                System.out.println("sum=" + sum);
        }
}
```

Statement	Sum	count
1	0	1
While loop_1	0+1 = 1	2
While loop_2	1+2 = 3	3
While loop_3	6	4
While loop_4	10	5
While loop_5	15	6
While loop_6	21	7
While loop_7	28	8
While loop_8	36	9
While loop_9	45	10
While loop_10	55	11
While loop stops (count>10)		

Counter Controlled Repetitions

- Example: A class of 10 students took a quiz. The grades (integer 0 to 100) are available. Determine the class average grade for quiz.
- To solve this problem, repetition is required to input 10 grades – one at a time
- For this we need **counter-controlled repetition**
- This technique uses a control variable (say counter) to control the number of times a set of statements will execute
- Counter-Controlled repetition is also called definite repetition as the number of repetitions is known before the loop begins execution.
- In the example, we know that 10 repetitions are required.

Pseudocode for Avg. Grade Example

1 Set total to zero

2 Set grade counter to one

3

4 While grade counter is less than or equal to ten

5 Prompt the user to enter the next grade

6 Input the next grade

7 Add the grade into the total

8 Add one to the grade counter

9

10 Set the class average to the total divided by ten

11 Print the class average

(Program Demo – Grade Book)

Sentinel Controlled Repetition

- Example:

  Consider class average program that processes grades on arbitrary number of students

- In this example, it is not known that how many grades (marks) user will enter during program execution
- It is not known when to stop the input of grades
- This can be solved using special value called sentinel value (signal value, dummy value or flag value) to indicate **“end of data entry”**
- **The user enters all grades and then enters sentinel value to endicate that no more grades will be entered**
- Sentinel controlled repetition are called indefinite repetition as the number of repetitions is not known before the loop begins execution

Pseudocode for class average program – sentinel controlled

1 Initialize total to zero
2 Initialize counter to zero
3
4 Prompt the user to enter the first grade
5 Input the first grade (possibly the sentinel)
6
7 While the user has not yet entered the sentinel
8 Add this grade into the running total
9 Add one to the grade counter
10 Prompt the user to enter the next grade
11 Input the next grade (possibly the sentinel)
12
13 If the counter is not equal to zero
14 Set the average to the total divided by the counter
15 Print the average
16 else
17 Print "No grades were entered”

Program Demo

Nested Control Statements

- Writing one control statement within another is called as nested control statements.
- Example:

  A college offers a course that prepares students for an exam. Say, 10 students completed the course and gave exam. The college wants to know how well the students did. A list is given, where in front of each student 1 is written is passed and 0 if failed. Write a program to determine hoe many students passed and how many failed

Pseudocode for example – nested control statements

```
1 Initialize passes to zero
2 Initialize failures to zero
3 Initialize student counter to one
4
5 While student counter is less than or equal to 10
6           Prompt the user to enter the next exam result
7           Input the next exam result
8
9           If the student passed
10                      Add one to passes
11          Else
12                      Add one to failures
13
14          Add one to student counter
15
16 Print the number of passes
17 Print the number of failures
18
19 If more than eight students passed
20 Print “Bonus to instructor !“
```

(program demo)

Compound Assignment Operators

- The compound assignment operators abbreviate assignment expressions
- Statements like:

  variable = variable operator expression;

  where operator is one of the binary operators +, -, *, / or %

Can be written as:

variable operator = expression;

- Example:

  c = c + 3 can be written as **c += 3**

Compound Assignment Operators

+=	**c += 7**	**c = c + 7**
-=	d -= 4	d = d – 4
*=	e *= 5	e = e * 5
/=	f /= 3	f = f / 3
%=	g %= 9	g = g % 9

Increment and Decrement Operators

- Java provides two unary operators for adding 1 to or subtracting 1 from value of a variable
- These are unary increment (++) and decrement (--) operators
- An increment or decrement operator that prefix to a variable is referred to as prefix increment or prefix decrement operator.
- An increment or decrement operator that is postfixed to a variable is referred to as postfix increment or postfix decrement operator.

Increment and Decrement Operators

++	Prefix increment	++a	Increment *a* by 1, then use the new value of a in the expression
++	Postfix increment	a++	Use the current value of *a* in the expression, then increment it by 1
--	Prefix decrement	--a	decrement *a* by 1, then use the new value of a in the expression
--	Postfix decrement	a--	Use the current value of *a* in the expression, then decrement it by 1

Increment and Decrement Operators

```
1 // Increment.java
2 // Prefix increment and postfix increment operators.
3
4 public class Increment
5 {
6               public static void main( String args[] )
7               {
8                               int c;
9
10                              // demonstrate postfix increment operator
11                              c = 5; // assign 5 to c
12                              System.out.println( c ); // print 5
13                              System.out.println( c++ ); // print 5 then postincrement (c=6)
14                              System.out.println( c ); // print 6
15
16                              System.out.println(); // skip a line
17
18                              // demonstrate prefix increment operator
19                               c= 5; // assign 5 to c
20                              System.out.println( c ); // print 5
21                              System.out.println( ++c ); // preincrement then print 6
22                              System.out.println( c ); // print 6
23
24              } // end main
25
26 } // end class Increment
```

output

```
5
5
6

5
6
6
```

Primitive Types

- There are eight primitive data types in Java
- Java requires all the variables to have a type
- So, Java is called **strongly typed language**
- Primitive type variables declared as instance variables of a class are automatically initialized (if not initialized by users)
- Instance variables of type char, byte, short, int, long, float and double are initialized to zero by default
- Instance variables of type boolean are initialized to false by default
- C and C++ programmers frequently have to write separate programs to support different computer platforms, because the primitive types are not guaranteed to be identical across platforms (memory size may change)
- Designers of Java wanted to ensure portability, so they used internationally recognized standards for character format (UNICODE)

Essentials of Counter Controlled Repetition

- A **control variable** (loop counter)
- The **initial value of control variable**
- The **increment (or decrement)** by which control variable is modified each time through the loop
- The **loop continuation condition** that decides if looping should continue

for repetition statement

- Repetition statement **for** can be used for **counter controlled repetitions**
- **Simple for loop example**

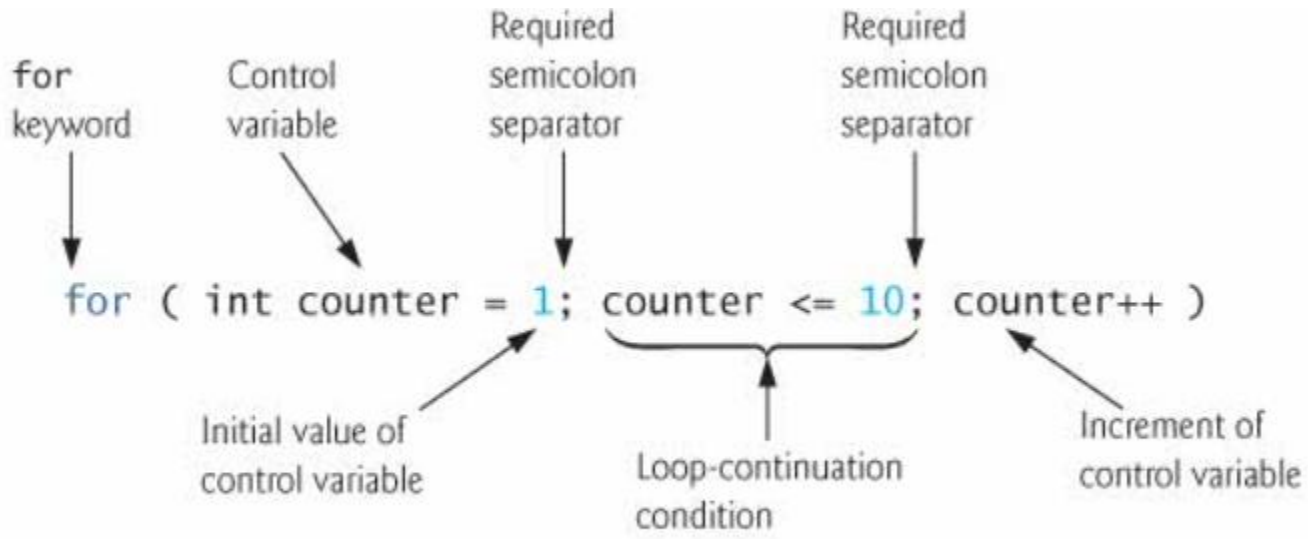
```
Public class ForCounter
{
        public static void main(String args[])
        {
                for (int counter=1; counter <= 10; counter++)
                        System.out.printf("%d ", counter);
        }
}
```

Output:
1 2 3 4 5 6 7 8 9 10

```
int counter = 1
while (counter <=10)
{
        System.out.printf("%d ", counter);
        counter++;
}
```

- When the for statement begins executing, the control variable counter is declared and initialized to 1.
- Next, the program checks the loop-continuation condition, counter <= 10, which is between the two required semicolons. (initial value of counter = 1)
- Therefore, the body statement displays control variable counter's value, namely 1.
- After executing the loop's body, the program increments counter in the expression counter++
- Then the loop-continuation test is performed again to determine whether the program should continue with the next iteration of the loop.
- At this point, the control variable value is 2, so the condition is still true: loop executes
- This process continues until the numbers 1 through 10 have been displayed and the counter's value becomes 11 (causes loop-continuation test to fail : for loop stops)
- The loop-continuation condition is counter <= 10.
- If the programmer incorrectly specified counter < 10 as the condition, the loop would iterate only nine times.
- This mistake is a common logic error called an **off-by-one error**.

for loop header



- **Initialization expression** names the loop's control variable and provides its initial value
- **Loop Continuation Condition** is the condition that determines whether the loop should continue executing
- **Increment** modifies the control variable's value

Placing Arithmetic expressions in for header

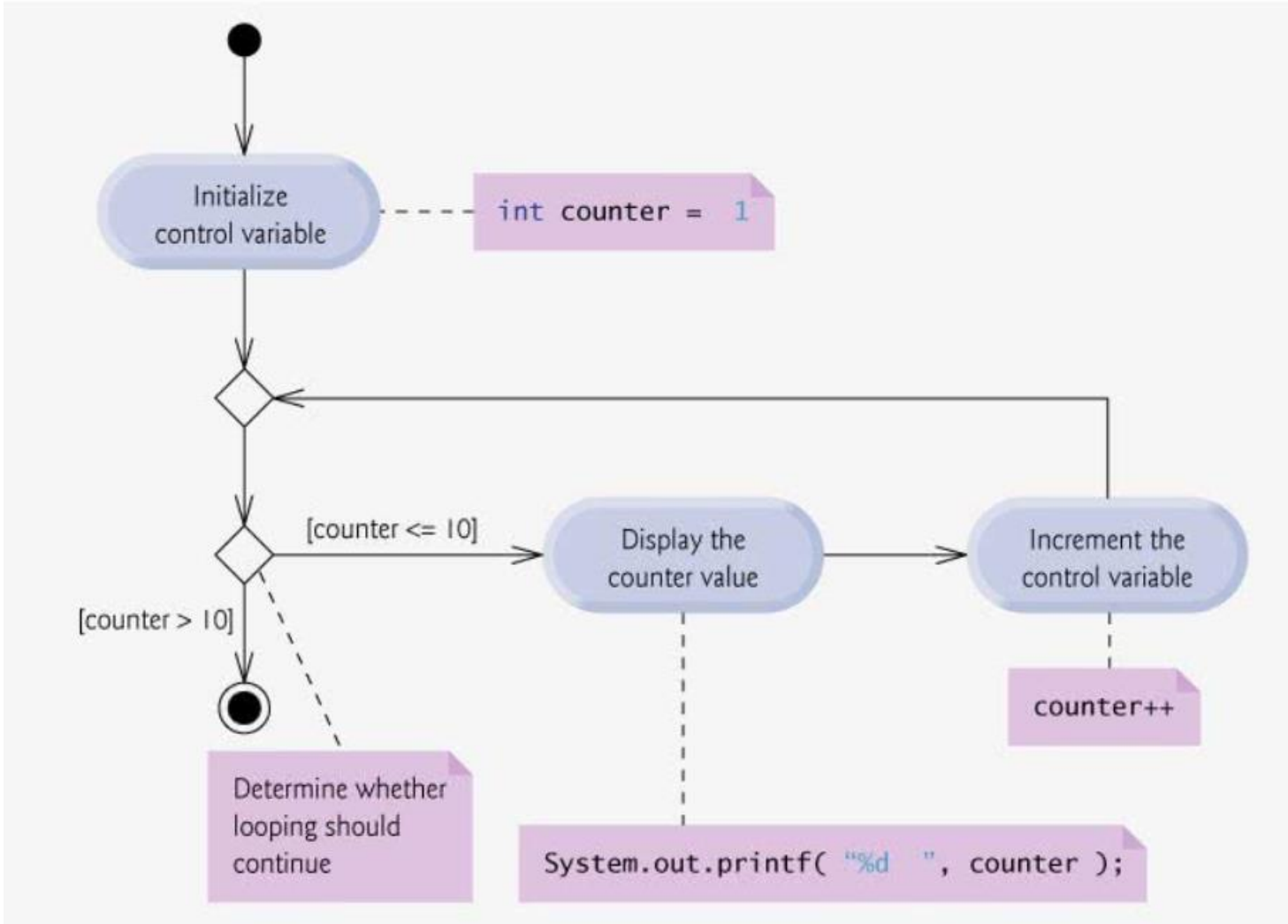
- The initialization, loop-continuation condition and increment portions of a for statement can contain arithmetic expressions.
- For example, assume that x = 2 and y = 10
- If x and y are not modified in the body of the loop, the statement

```
for ( int j = x; j <= 4 * x * y; j += y / x )
```

is equivalent to the statement

```
for ( int j = 2 ; j <= 80 ; j += 5 )
```

UML Activity Diagram for the for statement



Examples of for loop

```
for ( int i = 100; i >= 1 ; i-- )

for ( int i = 7; i <= 77; i += 7 )

for ( int i = 20; i >= 2; i -= 2 )

for ( int i = 2 ; i <= 20; i += 3 )

for ( int i = 99; i >= 0; i -= 11 )
```

program demo:

1. Summing the Even Integers from 2 to 20
2. Compound Interest Calculations

do…while Repetition Statement

- do…while repetition is similar to while statement
- The difference is: while statement may execute zero or many times
- The do..while statement executes at least one time (1 or many times)
- While loop tests condition at the beginning of loop, do..while tests after executing the loop body
- Syntax:

```
do {
    statement
    } while (condition);
```

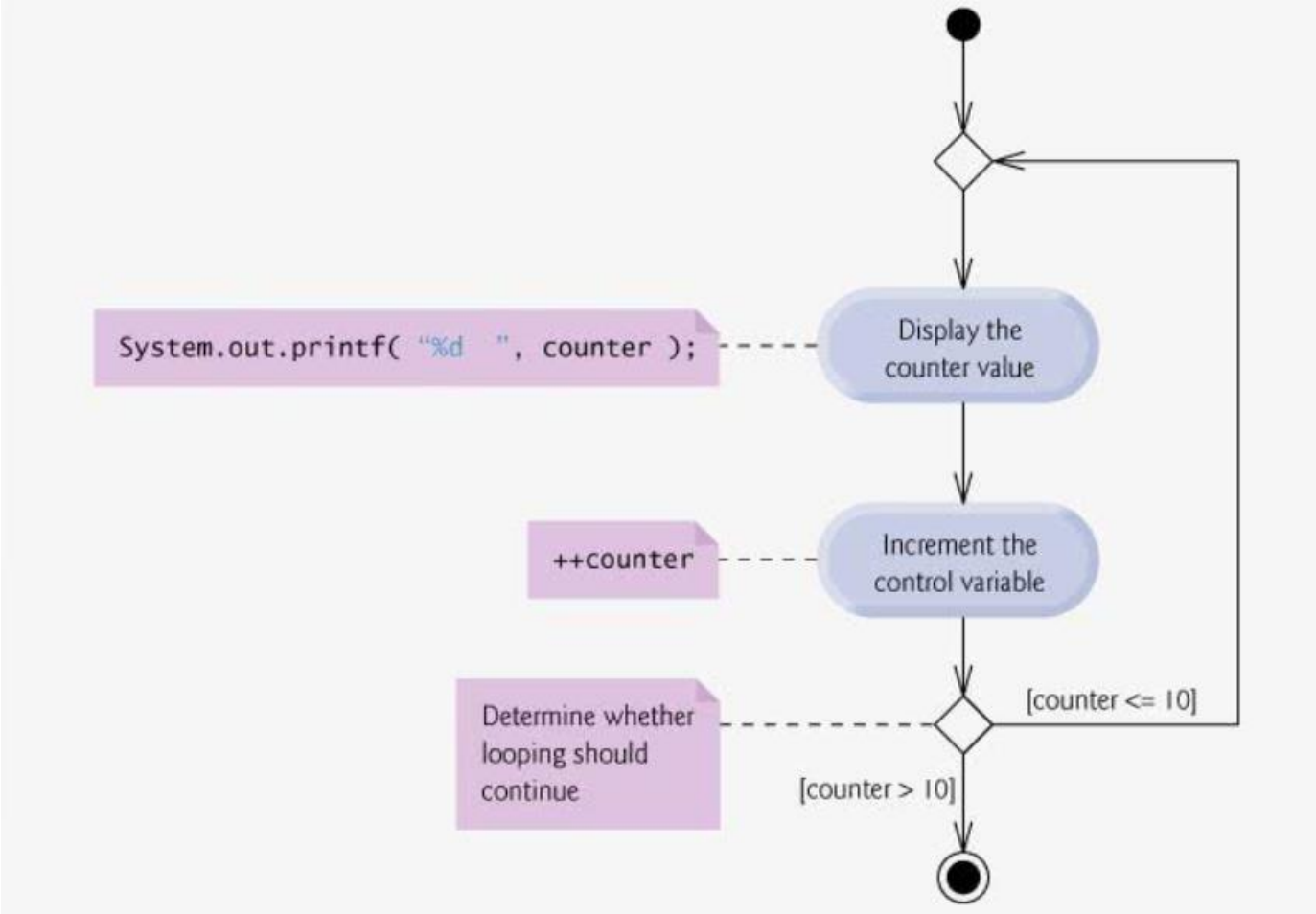
Example Program do...while

```
Public class DoWhileTest
{
        public static void main(String args[])
        {
                int counter = 1;
                do
                {
                        System.out.println(“%d ”, counter);
                        ++counter;
                } while (counter <= 10);
        }
}
```

Output:

1 2 3 4 5 6 7 8 9 10

UML Activity Diagram of Example (do..while)

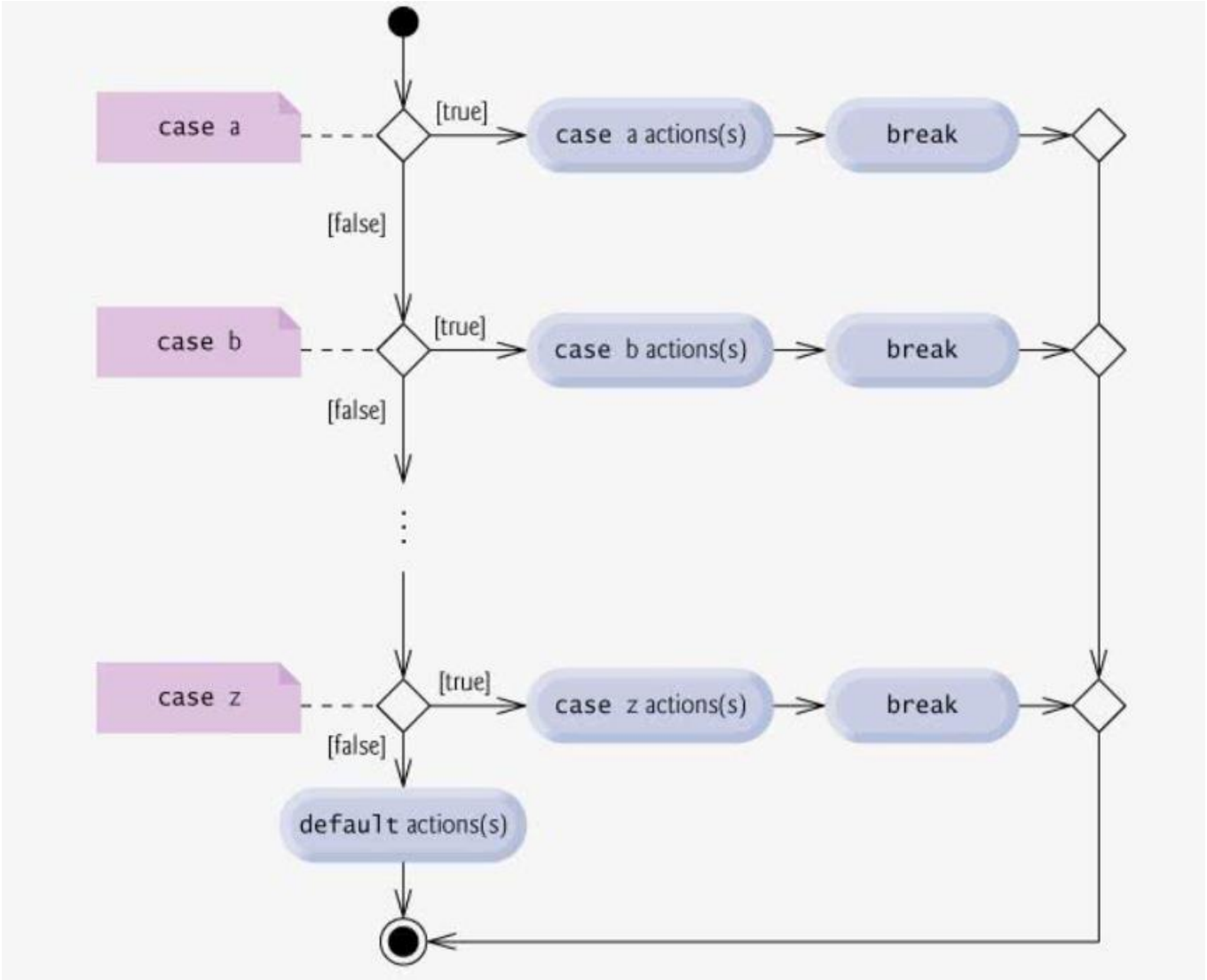


Switch-Multiple Selection Statement

- The switch multiple selection statement performs different actions based on the possible values of a constant integral expression of type byte, short, int or char.

- **Example Program:** Grade counter: Determines average grade of a class and also determines whether each entered grade is A, B, C, D, E or F and increments appropriate grade counter.

UML Activity Diagram of Switch Multiple Selection



Break and Continue

The break statement

- The break statement when executed in the while, for or do…while loop, skips the remaining iterations of the loop statement
- This makes the loop statement break the loop and proceed to the statement immediately after the loop statement

```
1 // BreakTest.java
2 // break statement exiting a for statement.
3 public class BreakTest
4 {
5    public static void main( String args[] )
6    {
7          int count;
8
9          for ( count = 1; count <= 10; count++ ) // loop 10 times
10         {
11            if ( count == 5 ) // if count is 5,
12                  break; // terminate loop
13
14            System.out.printf( "%d ", count );
15         } // end for
16
17         System.out.printf( "\nBroke out of loop at count = %d\n", count );
18    } // end main
19 } // end class BreakTest
```

Program : BreakTest

Output

1 2 3 4
Broke out of loop at count = 5

Continue Statement

- The continue statement, when executed in a while, for or do...while, skips the remaining statements in the loop body and proceeds with the next iteration of the loop.
- In while and do...while statements, the program evaluates the loop-continuation test immediately after the continue statement executes.
- In a for statement, the increment expression executes, then the program evaluates the loop-continuation test.

Program : ContinueTest

```
1 // ContinueTest.java
2
3 public class ContinueTest
4 {
5           public static void main( String args[] )
6           {
7               for ( int count = 1; count <= 10; count++ ) // loop 10 times
8               {
9                       if ( count == 5 ) // if count is 5,
10                                  continue; // skip remaining code in loop
11
12                      System.out.printf( "%d ", count );
13              } // end for
14
15              System.out.println( "\nUsed continue to skip printing 5" );
16          } // end main
17 } // end class ContinueTest
```

Output

1 2 3 4 6 7 8 9 10
Used continue to skip printing 5

Logical Operators

- Logical Operators are used to combine multiple conditions

Conditional AND (&&)

- Example :

  if (gender==“FEMALE” && age >= 65)

- This statement contains two simple conditions
- The condition *gender==“FEMALE”* compares variable gender with constant “FEMALE” and the condition *age >= 65* evaluates whether the person is senior citizen
- The if condition is true only if both the conditions are true

Conditional AND (&&) Truth Table

Expression1	Expression2	Expression1 && Expression2
False	False	False
False	True	False
True	False	False
True	True	True

CONDITIONAL OR (||)

- If we wish to ensure that either or both of two conditions are true before we choose a certain path of execution.
- In this case, we use the **||** (conditional OR) operator, as in the following program segment:

```
if ( ( semesterAverage >= 90 ) || ( finalExam >= 90 ) )
        System.out.println ( "Student grade is A" );
```

- This statement also contains two simple conditions.
- The condition semesterAverage >= 90 evaluates to determine whether the student deserves an A in the course semester Average.
- The condition finalExam >= 90 evaluates to determine whether the student deserves an A in the course because of marks in final exam.
- The if statement then considers the combined condition

```
( semesterAverage >= 90 ) || ( finalExam >= 90 )
```

It gives the student an A if either or both of the simple conditions are true.

Conditional OR (||) Truth Table

Expression1	Expression2	Expression1 && Expression2
False	False	False
False	True	True
True	False	True
True	True	True

Short-Circuit Evaluation of Complex Conditions

- The parts of an expression containing && or || operators are evaluated only until it is known whether the condition is true or false.
- Hence, evaluation of the expression

(gender == FEMALE) && (age >= 65)

- stops immediately if gender is not equal to FEMALE (i.e., the entire expression is false) and continues if gender is equal to FEMALE (i.e., the entire expression could still be True if the condition age >= 65 is true).

- This feature of conditional AND and conditional OR expressions is called **short-circuit evaluation**.

Boolean Logical AND (&) and Boolean Logical OR (|) Operators

- The **boolean logical AND** (&) and **boolean logical inclusive OR** (|) operators work identically to the && (conditional AND) and || (conditional OR) operators, with one exception:
- The boolean logical operators always evaluate both of their operands (i.e., they do not perform short-circuit evaluation).
- Therefore, the expression

  (gender == 1) & (age >= 65)

  evaluates age >= 65 regardless of whether gender is equal to 1.

- This is useful if the right operand of the boolean logical AND or boolean logical inclusive OR operator has a required **side effect (**a modification of a variable's value).
- For example, the expression

  (birthday == true) | (++age >= 65)

  guarantees that the condition ++age >= 65 will be evaluated. Thus, the variable age is incremented in the preceding expression, regardless of whether the overall expression is true or false.

Boolean Logical Exclusive OR (^)

- A simple condition containing the **boolean logical exclusive OR (^)** operator is true if and only if one of its operands is TRue and the other is false.
- If both operands are true or both are false, the entire condition is false.
- Truth Table:

Expression1	Expression2	Expression1 && Expression2
False	False	False
False	True	True
True	False	True
True	True	False

Logical Negation (!) Operator

- The **! (logical NOT**, also called **logical negation** or **logical complement**) operator enables a programmer to "reverse" the meaning of a condition.
- The logical negation operator is a unary operator that has only a single condition as an operand.
- The logical negation operator is placed before a condition

- Example:

```
if ( ! ( grade == sentinelValue ) )
    System.out.printf( "The next grade is %d\n", grade );
```

which executes the printf call only if grade is not equal to sentinelValue.

- The parentheses around the condition grade == sentinelValue are needed because the logical negation operator has a higher precedence than the equality operator.
- The previous statement may also be written as follows:

```
if ( grade != sentinelValue )
    System.out.printf( "The next grade is %d\n", grade );
```

Expression	!Expression
False	True
True	False